



Simulador de Detección de Interbloqueo (deadlock)

Propósito

El propósito de este proyecto es desarrollar un simulador que implemente un algoritmo de detección de interbloqueo (deadlock) en sistemas operativos. Los objetivos específicos son:

1. **Entender** y **explicar** el funcionamiento del algoritmo de detección de interbloqueo.
2. **Diseñar** y **programar** un simulador que permita visualizar el proceso de detección de interbloqueo.
3. **Probar** el simulador con diferentes escenarios para verificar su correcto funcionamiento.
4. **Documentar** el diseño y el uso del simulador.

Trasfondo

“La comprobación de si hay interbloqueo se puede hacer con tanta frecuencia como una vez por cada petición de recurso o, con menos frecuencia, dependiendo de la probabilidad de que ocurra un inter-bloqueo. Realizar la comprobación por cada petición de recurso tiene dos ventajas: conlleva una detección temprana y el algoritmo es relativamente sencillo debido a que está basado en cambios graduales del estado del sistema. Por otro lado, estas comprobaciones frecuentes consumen un considerable tiempo del procesador.

Un algoritmo usual para la detección del interbloqueo es el que se describe en [COFF71]. En el mismo se utilizan la matriz de Asignación y el vector de Disponibles descritos en la sección previa. Además, se define una matriz de solicitud S tal que S_{ij} representa la cantidad de recursos de tipo j solicitados por el proceso i . El algoritmo actúa marcando los procesos que no están en un interbloqueo. Inicialmente, todos los procesos están sin marcar.” Sistemas Operativos, William Stallings

El algoritmo se puede encontrar con más detalle en el libro de William Stallings en la página 275 aproximadamente o el Silberschatz en la página 233 aproximadamente.

Consideraciones de implementación:

El formato de ejecución de su programa debe ser el siguiente:

./deteccion -m d <nombre del archivo>

El archivo contiene un snapshot del flujo de recursos y ejecución de N cantidad de procesos, dicho archivo está definido por el siguiente formato:



N recursos - Nombrar a partir de la A mayúscula.

Vector recursos totales, este vector tiene el tamaño de N

Vector recursos disponibles, este vector tiene el tamaño de N

M procesos – Nombrar todos por P, los números van del [0,M-1]

Vector de recursos asignados | Vector de recursos totales para ejecución | Prioridad

Donde la prioridad va del 1 hasta el 10 siendo 1>10.

Ejemplo:

```
5
7 3 5 8 1
1 2 3 4 0
4
2 1 0 1 1 | 3 1 0 3 1 | 1
3 0 1 1 0 | 3 2 0 2 1 | 3
1 0 1 1 0 | 7 3 1 0 0 | 2
0 0 0 1 0 | 1 1 1 1 1 | 6
```

NOTA: El input se define como vectores para hacer más sencillo el procesamiento y almacenamiento de datos del archivo. Sin embargo, la estructura de datos de almacenamiento que se va a manipular en la implementación del algoritmo de detección de interbloqueo posee una estructura de tipo matriz, definida de la siguiente manera:



	R1	R2	R3	R4	R5
P1	0	1	0	0	1
P2	0	0	1	0	1
P3	0	0	0	0	1
P4	1	0	1	0	1

Matriz de solicitud S

	R1	R2	R3	R4	R5
P1	1	0	1	1	0
P2	1	1	0	0	0
P3	0	0	0	1	0
P4	0	0	0	0	0

Matriz de asignación A

R1	R2	R3	R4	R5
2	1	1	2	1

Vector de recursos

R1	R2	R3	R4	R5
0	0	0	0	1

Vector de disponibles

Figura 6.10. Ejemplo de detección del interbloqueo.

Salida:

Si el algoritmo encuentra interbloqueo, va a imprimir los procesos bloqueados, Ejemplo:

Interbloqueo detectado

Procesos involucrados P1 P3

Recursos disponibles 2 3 1 4 0

Si encuentra el interbloqueo, se debe dar la opción al usuario de terminar la simulación después de encontrar interbloqueo o terminar los procesos 1 a 1 hasta resolver el interbloqueo. por cada proceso terminado volver a correr la simulación y así sucesivamente, hasta que los procesos “finalicen” correctamente, los procesos terminados a la fuerza deben ser clasificados como procesos terminados.

Si hay más de un proceso bloqueado, el simulador debe dar las siguientes tres opciones al usuario:

-Aleatorio

-Prioridad, donde se termina el proceso con menor prioridad.

-Input, donde 0 es el proceso 0, 1 es el proceso 1 2 es el proceso 2....



Si el algoritmo no encuentra interbloqueo. Imprimir el posible orden de ejecución y asignación de recursos. Por cada proceso debe imprimir su estado si finalizo su ejecución o fue terminado para apropiarse de sus recursos. Ejemplo:

No se detectó interbloqueo

Procesos ejecutados:

P0 Finalizado

P1 Terminado

P2 Finalizado

P3 Finalizado

P4 Finalizado

Orden de ejecución:

P3, P1, P0, P2, P4

Nota: Si un proceso fue terminado, tiene que indicarse que fue terminado para continuar con la ejecución, en el orden de ejecución en el que se tomó la decisión de terminarlo ejemplo:

Orden de ejecución:

P3, P1 Terminado, P0 Terminado, P2, P4

Entregables

Su proyecto debe contener al menos los siguientes archivos al momento de ser entregado para revisión:

- Sus archivos de código fuente en C (archivos .c) y cabeceras relacionadas (archivos .h) necesarios para compilar el proyecto. Es obligatorio implementar al menos 1 cabecera.
- Un archivo Makefile que permita compilar el proyecto. El Makefile debe contener al menos las siguientes reglas:



1. all: debe permitir compilar el proyecto completo y producir el programa ejecutable correspondiente.
 2. clean: debe eliminar los archivos intermedios de compilación que se produzcan (por ejemplo, los archivos .o) y el ejecutable del proyecto.
- Un informe de mínimo 4 páginas en formato .PDF que contenga una descripción del programa desarrollado. Debe describir como está estructurado el código fuente e identificar los puntos de su código fuente donde se implementaron cada funcionalidad. Se debe anexar cada descripción/explicación con sus respectivas capturas. Además, se deben anexar casos de prueba, su estructura y su resultado en la ejecución de su solución.
 - Anexar los archivos de sus casos de prueba con resultados reales.

Consideraciones Generales:

- El proyecto deberá realizarse en grupo de 2 (dos) personas máximo.
- Para compilar el proyecto creado debe hacerse uso de un Makefile, el cual debe ser desarrollado por cada grupo, si el código es enviado sin el Makefile o el Makefile no funciona, el código no será evaluado.
- Los estudiantes son libres de organizar su código en diferentes cabeceras y secciones de código, es obligatorio el desarrollo y uso de cabeceras. De no cumplirse tendrá una penalización de 2 puntos.
- La implementación del proyecto será realizada utilizando el lenguaje de programación C, bajo un sistema Linux, de preferencia compatible con el sistema Ubuntu 24.04 LTS o derivados, debe ser compilable con cualquier versión actual de GCC. Ejemplo: gcc 13.3.0
- Usted deberá entregar el código fuente correspondiente al proyecto totalmente funcional y sin errores. Aquellos códigos que presenten errores de compilación no serán evaluados.
- No se aceptarán entregas fuera de la fecha y hora establecida.
- De acuerdo con lo establecido en la Ley de Universidades, las copias serán penalizadas con la nota mínima (0) para todos los involucrados.
- Se prohíbe el uso de IA generativas (LLM) para desarrollar el proyecto, de ser detectado la nota será severamente penalizada con la nota mínima (0).



Universidad Central de Venezuela
Facultad de Ciencias
Escuela de Computación
Sistemas Operativos (6004)
Proyecto #2



- El asunto del correo debe ser [SO]_[Proy2]_[Cedula1]_[Cedula2], si no cumple con el formato del asunto, la nota será severamente penalizada. De no cumplirse tendrá una penalización de 5 puntos.
- El código debe estar escrito de una manera clara, ordenada y bien documentada/comentada.
- Los entregables del proyecto deben ser enviados en un archivo .zip al correo laboratoriosisop@gmail.com.
- La fecha de entrega será determinada por el GDSO, se le avisará por el grupo de Telegram de la materia.

La distribución de puntos queda de la siguiente manera:

Informe - 4 pts

Defensa individual - 3 pts

Documentación/comentarios - 2 pts

Funcionalidad del código - 11 pts